Core Kotlin Mastery (Be fluent, not just familiar)

- **Null-safety & types:** `?`, `!!`, `lateinit`, smart-casts, `as?`, platform types (from Java), `Nothing`, `Unit`, `Any` vs `Object`, variance (`in`/`out`), star-projections.
- **Data/Sealed/Value classes:** `data class` copy/equals/hashCode nuances; `sealed` vs `sealed interface` for closed polymorphism; **value classes** (inline) for type-safety without allocation; when to prefer enums vs sealed.
- **Functions/FP:** higher-order fns, lambdas with receivers, `with`/`run`/`apply`/`also`/`let`, extension functions/properties, operator overloading, destructuring, tail recursion (usually avoid), partial application patterns.
- **Generics:** declaration-site vs use-site variance, `reified` type params in `inline` functions, generic constraints; function vs class variance tradeoffs.
- **DSLs:** infix functions, lambdas with receivers (`buildString`, Gradle Kotlin DSL, Ktor routing). Know how to make a tiny DSL and where readability breaks down.
- **Interop:** SAM conversions, `@JvmOverloads`, `@JvmStatic`, `@file:JvmName`, Kotlin collections vs Java mutability, checked exceptions, nullness on the JVM.
- **Modern language features to name-drop wisely:** context receivers (Kotlin 2.x), contracts (roughly), sealed hierarchies across modules, multi-catch alternatives (use `when`), `result` type patterns.

Coroutines, Flow, and Concurrency (the senior difference)

- **Structured concurrency:** prefer `coroutineScope`/`supervisorScope` over launching fire-and-forget; understand job hierarchies and cancellation propagation.
- **Contexts & dispatchers:** `Dispatchers.Default/IO/Main`, `withContext` vs `async/await`; avoid blocking calls; confine IO to `IO`.

- **Cancellation:** cooperative; check with `ensureActive()`, non-cancellable sections (`withContext(NonCancellable)`), and resource cleanup (`try/finally`).
- **Scopes:** who owns the scope and lifecycle? (Android: `viewModelScope`; server: per-request scope). Avoid leaking global `CoroutineScope`.
- **Flow vs Channel:** cold streams (`Flow`) with backpressure vs hot channels/`SharedFlow`/`StateFlow`; `buffer`, `conflate`, `flatMapLatest`, exception rules (`catch` placement).
- **Testing coroutines:** `runTest`, test schedulers/virtual time, `Turbine` for Flow assertions.

Be ready to answer: deadlocks with `withContext`, why `GlobalScope` is dangerous, how you throttle, debounce, and merge streams, and how you guarantee cancellation safety around DB/file ops.

Architecture & Design (Android or Backend)

- **Common stacks:**
    - **Android:** MVVM/MVI, repo pattern, DI with Hilt/Koin, Room, WorkManager, Compose, StateFlow vs LiveData, UI/State hoisting, unidirectional data flow, navigation & saved state.
    - **Backend (Ktor/Spring Boot with Kotlin):** routing DSLs, suspending handlers, non-blocking DB drivers, connection pools, structured logging, resilience (retries with jitter, circuit breakers), serialization (kotlinx.serialization/Jackson), DI (Koin/Spring).
- **Boundaries & contracts:** sealed results (`Success`/`Failure`), error mapping, DTO ↔ domain mappers, feature modularization, versioned APIs.
- **Concurrency model end-to-end:** do not expose `Flow`/`suspend` across layers without a reason; transform at boundaries, keep domain pure.

Testing, Quality, and Tooling

- **Unit tests:** kotest / JUnit5, fake vs mock, property-based tests; contract tests for DSLs.
- **Coroutine/Flow tests:** `runTest`, virtual time, Turbine.
- **Android:** Robolectric vs instrumentation, Compose testing (semantics).
- **Backend:** testcontainers for DBs & Kafka; contract tests for HTTP (Pact).
- **Static analysis:** Detekt, ktlint, Kover (coverage), Gradle Kotlin DSL hygiene.
- **Builds/CI:** Gradle configuration cache, parallelism, version catalogs, multiplatform targets.

Performance & Reliability

- **Hot spots:** allocations in lambdas/boxing, unnecessary `toList()`, `copy()` churn for immutable data, `map`/`flatMap` chains vs imperative loops when profiling.
- **Coroutines perf:** dispatcher hops, `withContext` overuse, `buffer` size tuning, `Channel` capacity.
- **JVM details:** escape analysis, inlining (`inline` vs `noinline/crossinline`), string handling, GC pressure; prefer value classes for IDs.
- **Android:** recomposition costs, `remember` keys, `derivedStateOf`, lists (`LazyColumn` diffing), image loading.
- **Server:** connection pool sizing, timeouts, backpressure, structured logging with MDC, metrics (Micrometer), tracing (OpenTelemetry).

Security & Robustness

- **Serialization safety:** unknown fields, polymorphic serialization, `@Serializable` defaults.

- **Input validation:** value classes, sealed hierarchies, non-nullable boundaries.
- **Secrets & config:** avoid hard-coding, use KMS/keystore; .properties vs env; Gradle buildConfig for variants.
- **Error strategy:** typed errors, retry policies (exponential backoff + jitter), idempotency keys, compensating actions.

Kotlin Multiplatform (KMP) Talking Points

- **Expect/actual:** platform abstractions; share domain/network/cache; platform-specific UI (Compose Multiplatform/SwiftUI).
- **Limitations:** native memory model, binary compatibility, third-party libs; testing strategy across targets.
- **When to use:** shared business logic across Android/iOS/desktop/web.

System Design with Kotlin (Server example)

Prompt to practice: *Design a rate-limited URL shortener with Kotlin (Ktor), Postgres, and Redis.*

- API: create/resolve; throttling per IP with Redis; durable keys in Postgres; background cleanup job; metrics/tracing.
- Concurrency: suspending DB ops; backpressure via `Flow` for streaming analytics; coroutine scopes per request.
- Data model: `ShortLink(id: ValueClass, longUrl, createdAt, ownerId)`; unique index; TTL for anonymous links.

Idiomatic Patterns & Anti-patterns

- Prefer **sealed results** over exceptions for domain flow; use exceptions for truly exceptional/infra failures.
- Avoid `!!`; design types that don't need it.
- Avoid "God" scopes and `GlobalScope.launch`.
- Distinguish **cold** flows from **hot**; don't expose `MutableStateFlow` outside modules.
- Keep **DI graphs** shallow and constructor-inject dependencies.

Behavioral (Senior signals)

- **Leadership:** mentoring, code review philosophy, enabling autonomy, setting quality bars (linting, testing, docs).
- **Ownership:** oncall/incident response; postmortems with action items you drove.
- **Product sense:** tradeoffs you made to hit dates; slicing features; saying "no" with data.
- **Cross-team:** aligning contracts, killing flaky tests, setting SLIs/SLOs.
- **STAR template:** Situation → Task → Actions → Results (+ your Learnings/Next steps).

Behavioral practice prompts

- "Tell me about a time you led a refactor that paid off."
- "Describe a production incident you owned—what permanently changed afterward?"
- "When did you push back on a requirement and why?"

Be ready to compare with exceptions, especially across async boundaries.

Deep-Dive Questions You Should Be Ready For

- How does **structured concurrency** prevent leaks vs manual job tracking?
- Difference between `map`, `flatMapConcat`, `flatMapMerge`, `flatMapLatest` in Flow and when to choose each.
- Explain **reified** generics and a real use in your codebase.
- When to use **value classes**; pitfalls (boxing, type erasure at some boundaries).
- How would you **profile** a hot path in Kotlin (Android or server)? Specific tools and what you'd look for.
- Compose state hoisting vs two-way data binding; how you prevent **recomposition storms**.
- In Ktor/Spring, how you ensure **non-blocking IO** all the way down (DB drivers, HTTP clients).
- How do you implement **cancellation-safe** cleanup around DB transactions or file writes?

7-Day Brush-Up Plan (short & pragmatic)

- **Day 1–2:** Core Kotlin drills (generics, reified, sealed/value classes, DSLs).
- **Day 3:** Coroutines/Flow: build + test a small streaming pipeline; practice `runTest` + Turbine.
- **Day 4:** Pick your target (Android or Ktor/Spring). Do one feature end-to-end with DI, persistence, and tests.
- **Day 5:** System design: design a streaming service + rate limiting; write down APIs, data models, scaling.
- **Day 6:** Performance: run a profiler on your feature; optimize something; document wins.
- **Day 7:** Behavioral: craft 6 STAR stories (leadership, incident, refactor, cross-team, product tradeoff, mentoring). Rehearse out loud.

Day-Of Interview Checklist

- Be explicit about **ownership and lifecycles** of coroutine scopes.
- Name **tradeoffs and failure modes** unprompted.
- Write **idiomatic** Kotlin (extension fns, sealed results, value classes).
- Use **tests** to drive a tricky Flow/coroutine scenario.
- Tie choices to **operational excellence** (logging, metrics, tracing, SLOs).
- Close strong with questions about **quality bars**, **incident culture**, and **technical vision**.